

System Architecture Specification

Source: docs/ARCHITECTURE.md • Status: Verified & Synchronized

System Architecture

Product: Cubyntra

Organization: Necookie Labs

Document Version: 1.0

Status: V1 Release

Last Updated: 2026-09-26

1. Architectural Philosophy & Principles

Cubyntra is engineered according to five fundamental architectural tenets:

- Mathematical Ground Truth Over Graphics: The logical domain model (CubeState) is the absolute ground truth. The 3D Three.js rendering engine is purely a visual projection.
- Zero-Cloud Privacy Sandbox: Video streams and pixel buffers remain strictly confined to the local browser context.
- Unidirectional State Flow: State changes propagate strictly downward via the centralized Zustand store (useCubyntraStore), preventing split-brain or synchronization anomalies.
- Drift-Free Digital Twin: 3D layer rotations snap cubie coordinate vectors and rotation quaternions back to orthogonal integers after every turn.
- Decoupled Move Representation: Moves are represented as hardware-agnostic CubeMove objects, creating an extension point for physical robotics.

2. End-to-End Pipeline Architecture

flowchart TD

```
subgraph PERCEPTION [Perception Layer (Client-Side)]
  A[WebRTC Video Stream] --> B[Offscreen 2D Canvas]
  B --> C[Centered ROI Extraction]
  C --> D[3x3 Cell Sampling]
  D --> E[Trimmed-Mean Pixel Aggregation]
  E --> F[sRGB to HSV & CIELAB Space]
  F --> G[Lighting-Adaptive Classifier]
  G --> H[Temporal Stability Buffer]
  H --> I[Consensus Reached]
  I --> J[Scanned Face Matrix]
end
```

```
subgraph DOMAIN [Cube Domain & Validation Layer]
  J --> K[6-Face State Reconstruction]
  K --> L[Physical Invariant Validator]
  L --> M[Check Sticker Counts & Centers]
  M --> N{Basic Valid?}
  N -- No --> O[Diagnostic Error Recovery]
  N -- Yes --> P{Parity Valid?}
  P -- Twist / Flip / Permutation Error --> M
  P -- Valid --> Q[Logical CubeState]
end
```

```
subgraph SOLVING [Algorithmic Solving Layer]
  Q --> R[Facelet Serializer URFDLB]
  R --> S[Herbert Kociemba Two-Phase Engine]
  S --> T[Raw Move Sequence]
end
```

```

    R --> S[Move Parser & HTM Normalizer]
    S --> T[Post-Solve Simulation Verifier]
    T -->|Verified Solved| U[Hardware-Agnostic CubeMove[]]
end

subgraph PRESENTATION [Visualization & Execution Layer]
    U --> V[Move Dispatcher / Planner]
    V --> W[Three.js 27-Cubie Digital Twin]
    V --> X[3D Directional Move Arrows]
    V --> Y[Interactive UI Ticker & Controls]
    W -->|Integer Orthogonal Snapping| Z[Drift-Free Visual Twin]
    V -. ->|Future Hardware Extension Point| HW[ESP32 / Physical Motor Controller]
end

```

3. Module Boundaries & Dependency Inversion

The project enforces strict directory boundaries and unidirectional dependencies:

```

src/
% % % cube/           (Pure Domain Models - Zero DOM or Three.js dependencies)
% % % vision/         (Computer Vision - MediaDevices, Canvas, Color Spaces)
% % % solver/         (Algorithmic Solver - Kociemba Two-Phase, HTM)
% % % three/          (3D Graphics Engine - Three.js WebGL, 27 Cubies, Snapping)
% % % stores/         (Zustand State Coordinator - UI, Flow, Telemetry)
% % % components/     (Modular React Presentation - Viewfinder, Controls, Background)

```

Dependency Direction Rules:

- `src/cube/` has zero external dependencies. It contains pure TypeScript models, math invariants, face rotations, and parity checks.
- `src/vision/` depends only on `src/cube/` for type definitions (`Face`, `CubeColor`, `ScannedFace`). It has zero Three.js dependencies.
- `src/solver/` depends only on `src/cube/` to serialize `CubeState` and verify solutions.
- `src/three/` depends on `src/cube/` for `CubeState` and `CubeMove` contracts. It receives states and animates them.
- `src/stores/` orchestrates cube, vision, solver, and three.
- `src/components/` connects UI events to `src/stores/`.

4. Computer Vision Isolation

The Computer Vision module is strictly decoupled from the UI rendering cycle:

- Video frames are captured and processed via a direct `requestAnimationFrame` loop on an offscreen canvas.
- Component state is only updated when classifications or stability values change, preventing high-frequency React re-renders.
- In headless testing or environments without a camera, `src/vision/mock.ts` provides synthetic scan fixtures that inject valid scramble states directly into the domain model.

5. Solver Isolation & Verification

The solver module (`src/solver/`) is encapsulated behind a clean contract:

```
export async function solveCube(state: CubeState): Promise<SolveResult>;
```

Before passing a state into the Kociemba two-phase search, `solveCube` runs `validateCubeState(state)`. After Kociemba returns a candidate sequence, `solveCube` simulates applying the moves on an internal deep-cloned `CubeState` using `applyMoves(state, moves)`. Only if `isCubeSolved(simulatedState) === true` does it return success.

6. Zero Floating-Point Drift Engine

In standard 3D engines, repeatedly applying Euler angles or quaternions leads to accumulated floating-point inaccuracies ($\sin(\theta)$, $\cos(\theta)$ rounding errors), resulting in misaligned stickers after 20–30 turns.

Cubyntra eliminates drift completely:

- Grouping: During animation, 9 cubies are attached to a temporary pivotGroup.
- Animation: Smooth easing rotates the pivot around the face axis.
- Detachment: Upon completion, cubies are detached back to rootCubeGroup.
- Integer Snapping: World coordinates are rounded to exact integers ($x, y, z \in \{-1, 0, 1\}$).
- Orthogonal Angle Snapping: Quaternions are snapped to multiples of $\pi / 2$.
- State Resynchronization: Cubie sticker colors are refreshed directly from the logical CubeState.

Result: 0.000% accumulation drift across thousands of rotations.

7. Future Hardware Extension Points

Cubyntra V1 produces a standardized array of CubeMove objects:

```
export interface CubeMove {  
  face: 'U' | 'D' | 'L' | 'R' | 'F' | 'B';  
  quarterTurns: 1 | -1 | 2; // 1 = 90° CW, -1 = 90° CCW, 2 = 180°  
  notation: string;  
  description: string;  
}
```

This structure serves as an extensible hardware dispatch abstraction:

```
flowchart LR  
  CubeMove["CubeMove[]"] --> Dispatcher[MoveDispatcher]  
  Dispatcher --> ThreeJs[ThreeJsExecutor (V1)]  
  Dispatcher --> HumanGuide[HumanGuideExecutor (V1)]  
  Dispatcher -. "FUTURE" .-> Serial[WebSerial / WebBluetooth]  
  Serial -. " " .-> ESP32[ESP32 Microcontroller]  
  ESP32 -. " " .-> Steppers[6x Stepper Motors]
```

The hardware pipeline is FUTURE / OUT OF SCOPE FOR V1.
